Diet Planner App

Ekrem Karakas, Suhani Kashyap, Camille Ferrell, Benjamin Foore

1. Motivation & Problem Statement

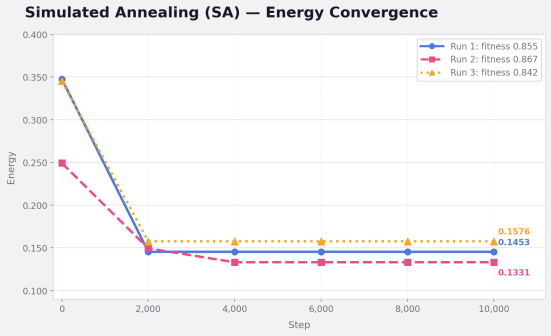
Balancing daily nutritional intake is challenging because foods vary widely in their macro- and micronutrient composition, making it difficult for individuals to meet dietary targets without exceeding or falling short in key nutrients. Most existing meal-planning tools generate static daily or weekly plans that assume perfect adherence, even though people routinely deviate by snacking, eating out, or adjusting portion sizes. When these deviations occur, popular apps such as MacroFactor primarily provide tracking and analytics rather than adaptive guidance that responds to what the user actually consumed. Similarly, services like Hungryroot offer personalized weekly grocery and recipe recommendations, but these suggestions remain fixed and do not update throughout the day as eating behavior changes. This creates a gap between planned and real-world dietary patterns, often leading to nutritional imbalance or unmet goals. Our project addresses this gap by developing a dynamic meal-planning system that recalculates the remaining nutritional budget after each logged meal. The system treats consumed foods as fixed constraints and re-optimizes the rest of the day using stochastic search methods such as simulated annealing and genetic algorithms. This reframes meal planning as a continuous multi-objective optimization problem that balances nutrition, variety, user preferences, and realistic meal compatibility. The core research question guiding this work is: **How can we design an adaptive meal-planning algorithm that updates throughout the day to recommend nutritionally balanced, realistic, and user-aligned meals after each logged food choice?**

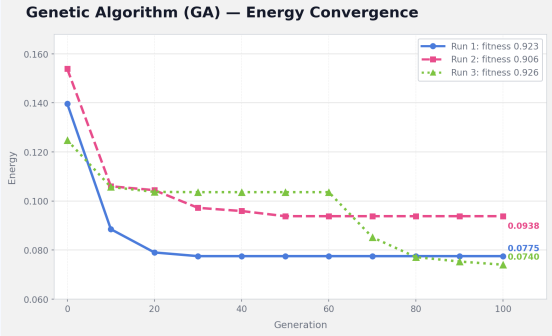
2. Approach

Our system follows a rolling-horizon optimization framework that updates meal recommendations dynamically based on what the user has already eaten. We begin by preprocessing two large datasets, USDA Foundation Foods and OpenFoodFacts, normalizing all entries to a per-100g basis, removing incomplete or unrealistic foods, filtering out non-standalone items, and capping categories to ensure variety, resulting in a cleaned dataset of roughly 10,000 foods. Logged meals are treated as fixed constraints, and the remaining nutrient budget is passed into a greedy initialization module that constructs a strong starting plan by sampling candidate foods, estimating optimal portion multipliers, and applying soft penalties to discourage repeated categories. Each meal plan is represented as a list of (food_index, multiplier) tuples, one per remaining meal slot. Then optimization is performed using either Simulated Annealing (SA) or a Genetic Algorithm (GA). SA refines a single candidate solution by applying neighbor moves like replacing a food, swapping two slots, or resizing a portion by a random delta taken from a uniform distribution between -0.5 and 0.5, and accepts worse states probabilistically based on a geometrically decaying temperature schedule. In comparison, GA evolves a population of meal plans using tournament selection, crossover, mutation, and elitism, enabling broader exploration of the search space and typically achieving higher final fitness. Both optimizers minimize an energy function defined as 1 - fitness, where the multi-objective fitness function evaluates nutrition accuracy, variety, meal compatibility, user preference, and follow-through likelihood. User preference is predicted using a Gaussian Naive Bayes classifier implemented from scratch with NumPy, trained on nutritional features so it can generalize to foods outside known categories. The system is implemented in Python using pandas and NumPy, with separate components for preprocessing, initialization, scoring, and optimization. We assume single-day planning and do not model grocery constraints, multi-day dependencies, or long-term dietary patterns, all of which can be explored in future extensions.

3. Key Results

Across all the experiments, the Genetic Algorithm (GA) demonstrated superior performance relative to Simulated Annealing (SA) in optimizing daily meal plans. Using greedy initialization, GA achieved final fitness scores of 0.923, 0.906, and 0.926 across three independent runs, corresponding to low energy values between 0.075 and 0.094. In contrast, SA converged to lower fitness values of 0.855, 0.867, and 0.842, with energy values between 0.133 and 0.158. Notably, SA had an early plateau around step 2000, indicating that the cooling schedule limited its ability to escape local minima despite additional iterations. Comparing the two methods further highlights this disparity: GA attained an average final fitness of 0.918, exceeding SA's average of 0.855 by approximately 7.4%. These results suggest that population-based search is more effective for this multi-objective optimization problem, as GA continues to explore various paths and avoids converging early. Additionally, GA-generated meal plans achieved higher variety and stronger meal-type compatibility, often reaching maximal sub-scores in these dimensions. SA performed well during the initial exploration phase but was less capable of refining solutions once the temperature decreased, consistent with expectations for single-path stochastic search. Overall, the findings validate the system's design choices and demonstrate that combining greedy initialization with GA yields the most reliable and nutritionally comprehensive meal plans.





4. Challenges & Lessons Learned

One challenge we encountered was that early versions produced unrealistic portion sizes, such as breakfasts that were disproportionately large or dinners that were implausibly small. This issue stemmed from the optimizer assigning extreme multipliers to foods with unusual calorie densities. We mitigated this issue by enforcing strict `portion_min` and `portion_max` bounds in both the greedy initializer and the optimization algorithms, which stabilized serving sizes across all meal slots. Another challenge involved incorporating user preferences in a way that generalized beyond simple category matching. Our initial rule-based approach proved too rigid, so we implemented a Gaussian Naive Bayes classifier trained on nutritional features to generate preference probabilities for all foods, significantly improving personalization. We also faced organizational challenges in coordinating preprocessing, initialization, scoring, and optimization modules, which required consistent data formats and shared configuration parameters. A key lesson learned was the importance of clear communication and division of responsibilities within the team, especially when multiple components needed to integrate seamlessly. More broadly, we found that multi-objective optimization requires careful tuning of scoring weights, as small adjustments can meaningfully alter algorithm behavior. These insights will guide any possible future extensions, particularly for multi-day planning and more complex dietary constraints.

5. Individual Contributions

Member	Primary Contributions	Approx. %
Ekrem Karakas	*Preprocessing Data, Simulated Annealing, Energy Function*	25%
Benjamin Foore	*Naive Bayes Classifier (user preference scoring), Greedy Initialization (initialize.py)*	25%
Camille Ferrell	*Genetic Algorithm, Fitness Function, Data set translation*	25%
Suhani Kashyap	*Data Collection, Result Plots, Report Writing*	25%

6. Repository & References

GitHub Repository: `https://github.com/4oore/CS4100-Group-Project`

Project Tracker: `https://github.com/users/4oore/projects/1`

Key References:

[1] Kirkpatrick, S., Gelatt, C. D., and Vecchi, M. P., "Optimization by Simulated Annealing," *Science*, 1983. Foundational paper introducing simulated annealing.

[2] Goldberg, D. E., "Genetic Algorithms in Search, Optimization, and Machine Learning," Addison–Wesley, 1989. Classic text on genetic algorithms and evolutionary search.

[3] Holland, J. H., "Adaptation in Natural and Artificial Systems," University of Michigan Press, 1975. Seminal work establishing the theory of evolutionary computation.